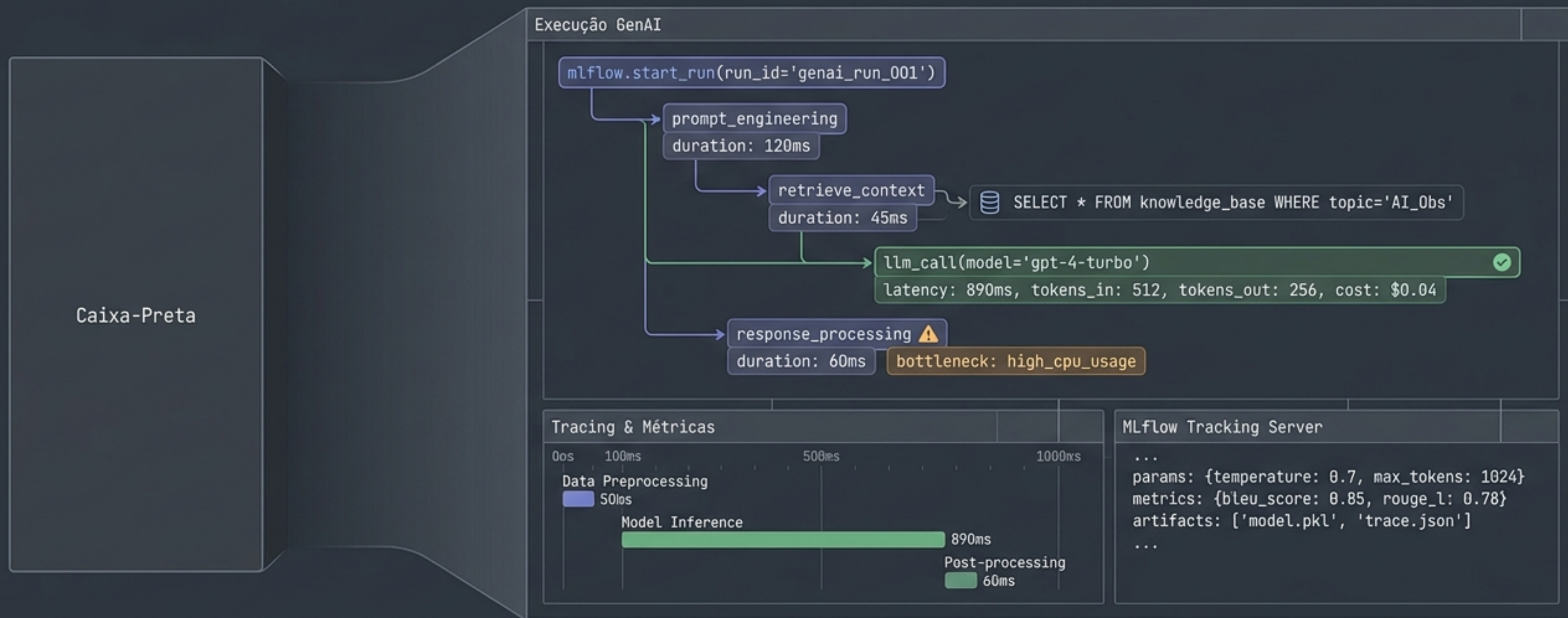


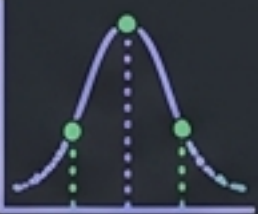
Observabilidade em Aplicações de IA: Da Caixa-Preta à Engenharia Precisa

[Status: Terminal Ready]

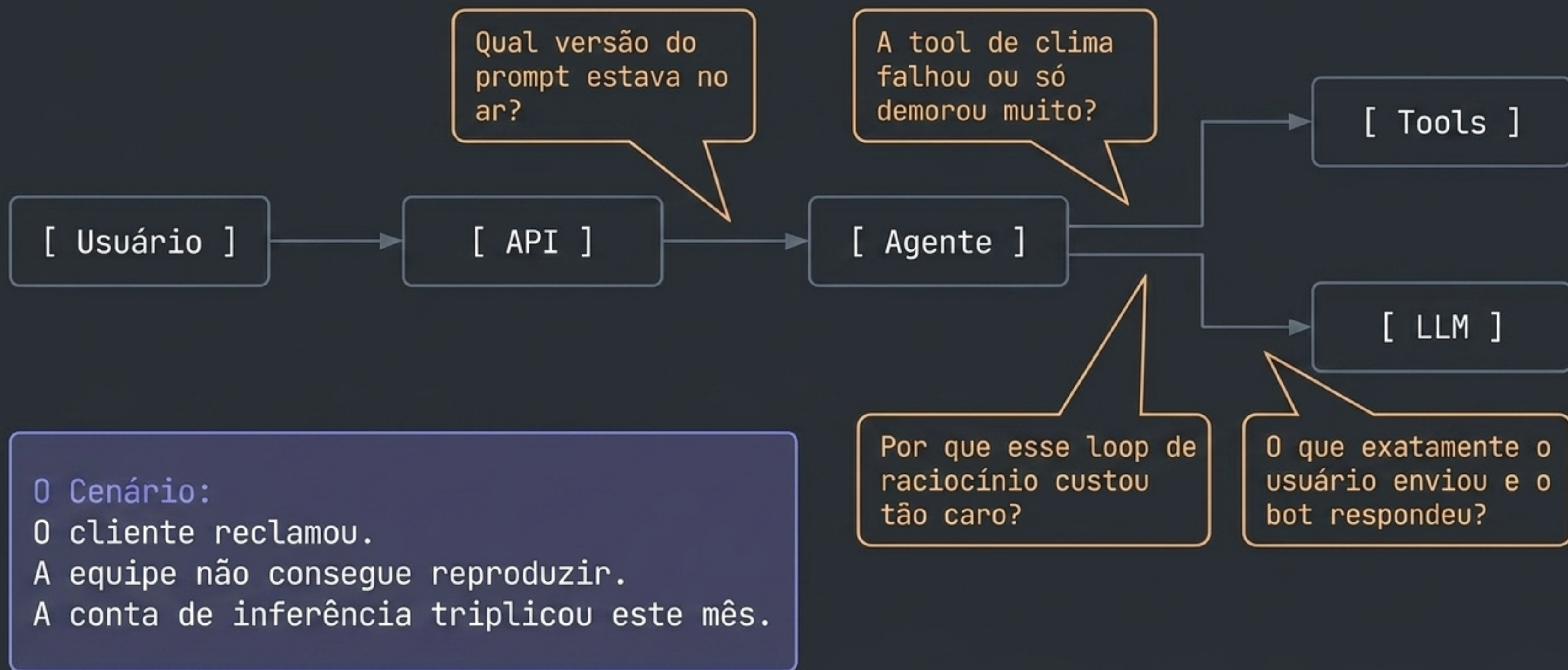
Workshop Prático: MLflow GenAI (1h)



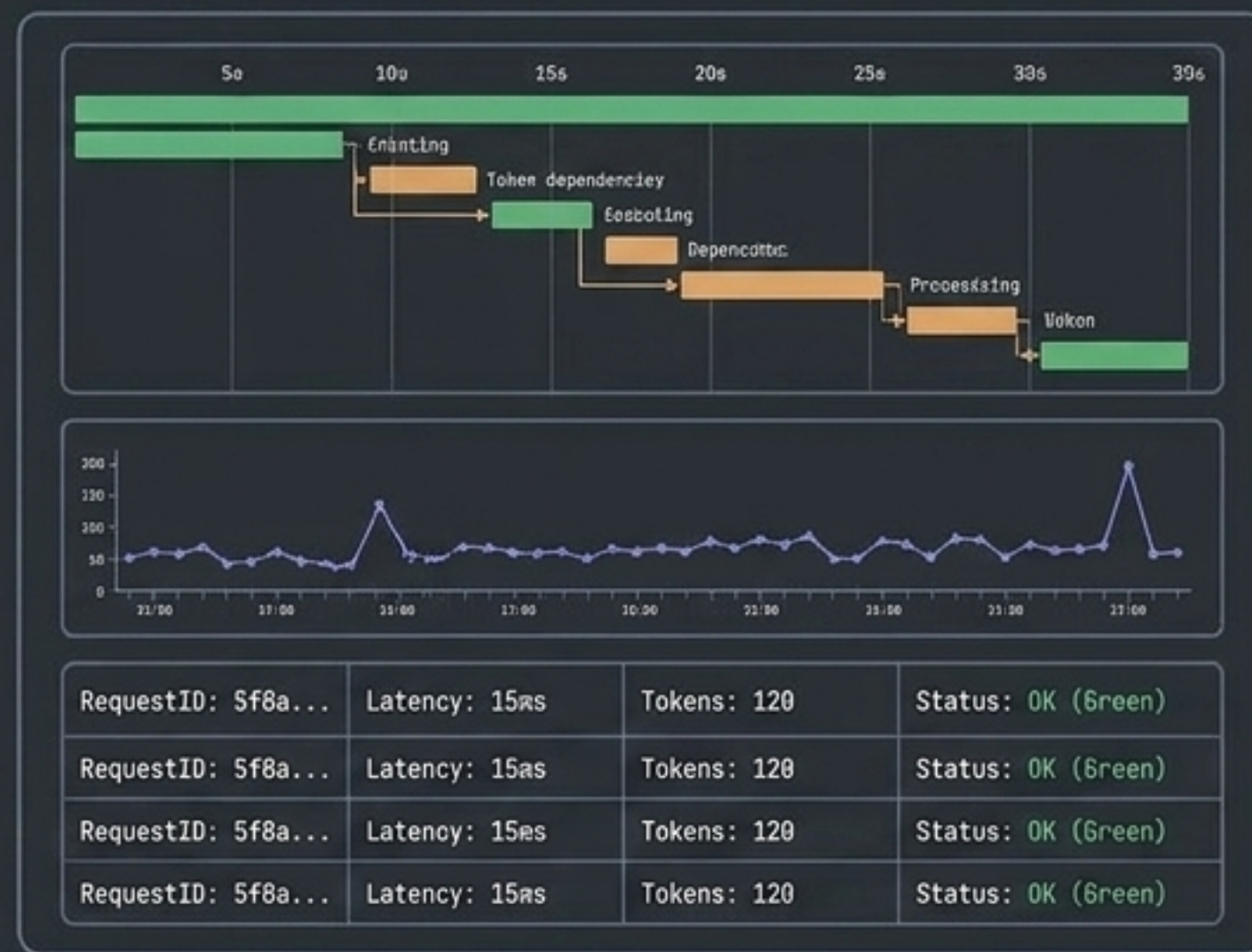
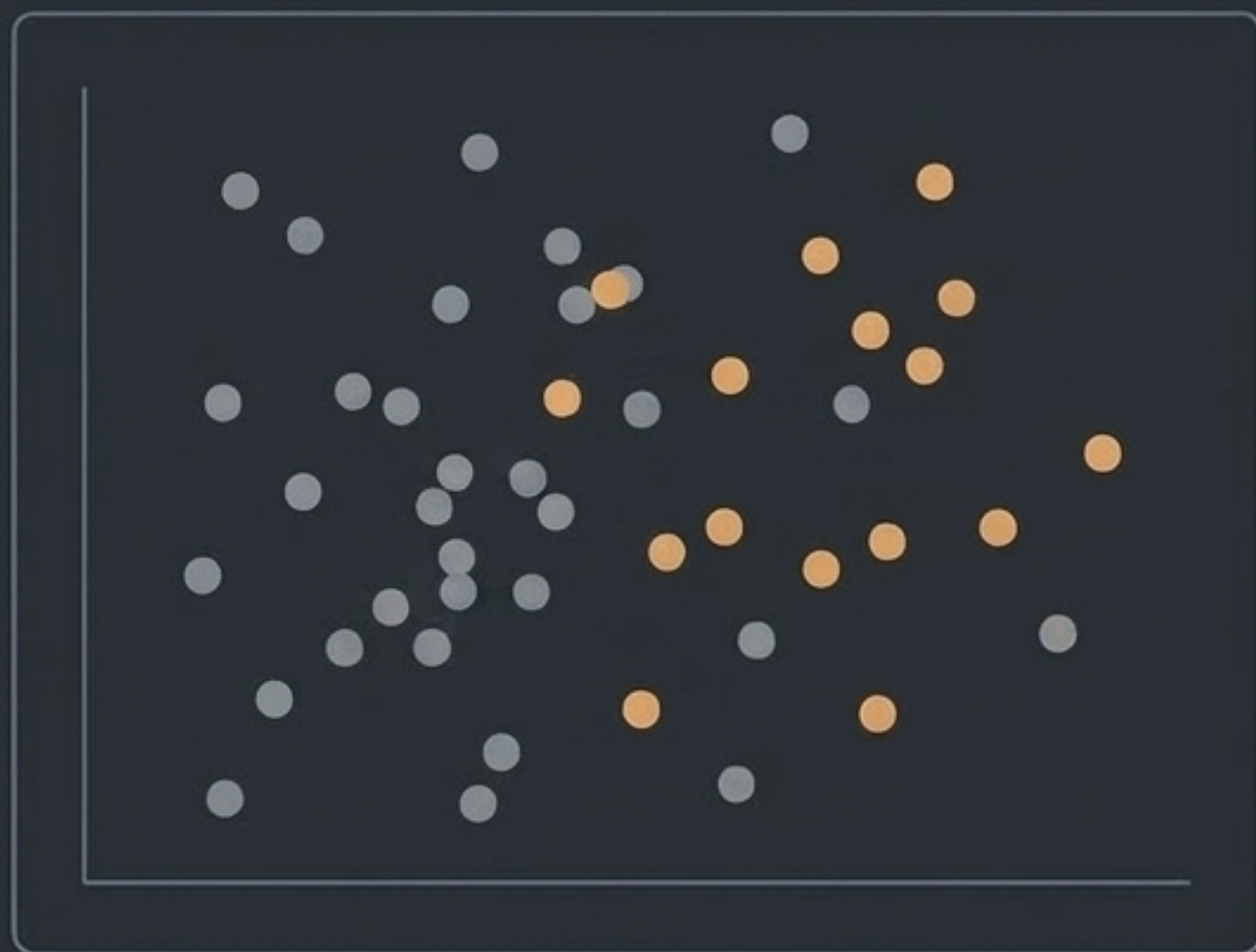
0 Fim do Determinismo (Por que as regras antigas quebram)

	Software Tradicional	Aplicações LLM
Resultado	Fixo e previsível. 	Não-determinístico a cada chamada. 
Modo de Falha	Erro 500 / Stacktrace imediato. 	HTTP 200 com alucinação completa (Falha silenciosa). 
Debug	Bug 100% reproduzível via testes. 	Caixa-preta iterativa. 
Custo	Fixo (Infraestrutura/Servidor). 	Variável e destrutivo (Multiplicado por tokens). 

0 Incidente das 14h32



Sem observabilidade, melhorar uma aplicação de IA vira chute. Com observabilidade, vira engenharia.



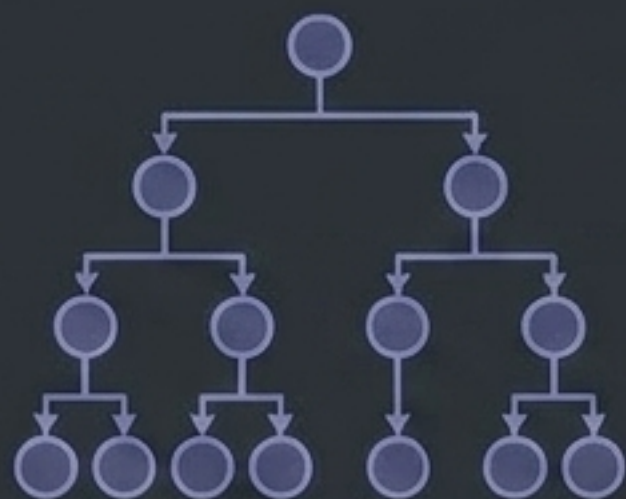
[TRACES]

[TOKENS/CUSTO]

[LATÊNCIA]

[QUALIDADE]

0 Framework: Os 4 Pilares da Observabilidade



1. Tracing (O Como)

Registra cada passo, input e latência da execução.



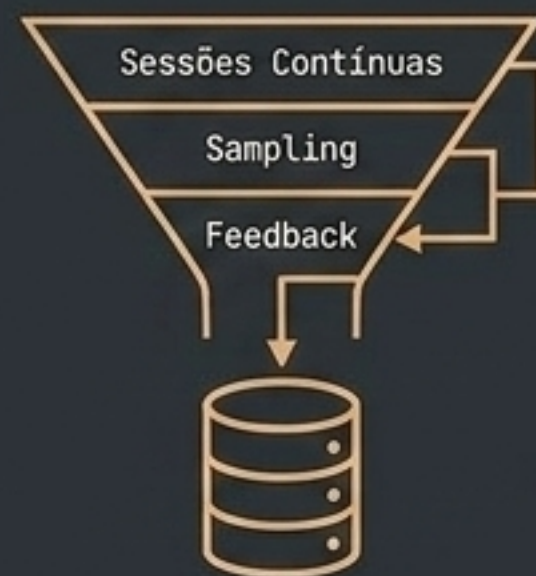
2. Custo & Tokens (O Quanto)

Consumo exato de tokens e impacto financeiro por chamada.



3. Avaliação (O Quão Bom)

Medição de qualidade de forma sistemática e contínua.



4. Produção (Em Escala)

Sampling inteligente, sessões contínuas e coleta de feedback humano.

1. Tracing: O Fim da Adivinhação

> make tracing

tracing_basics.py

```
1 import mlflow
2
3 # Auto-tracing com 1 linha
  mlflow.openai.autolog()
```

Nenhuma instrumentação manual necessária.
Captura imediata de inputs, outputs e latência.

Input

```
{
  'messages': [{ 'role': 'user',
                  'content': 'Summarize the article.' }],
  'model': 'gpt-4',
  'temperature': 0.7
}
```

Output

```
{ 'choices':
  [ { 'message': { 'role': 'assistant',
                  'content': 'The article discusses...' } } ]
}
```

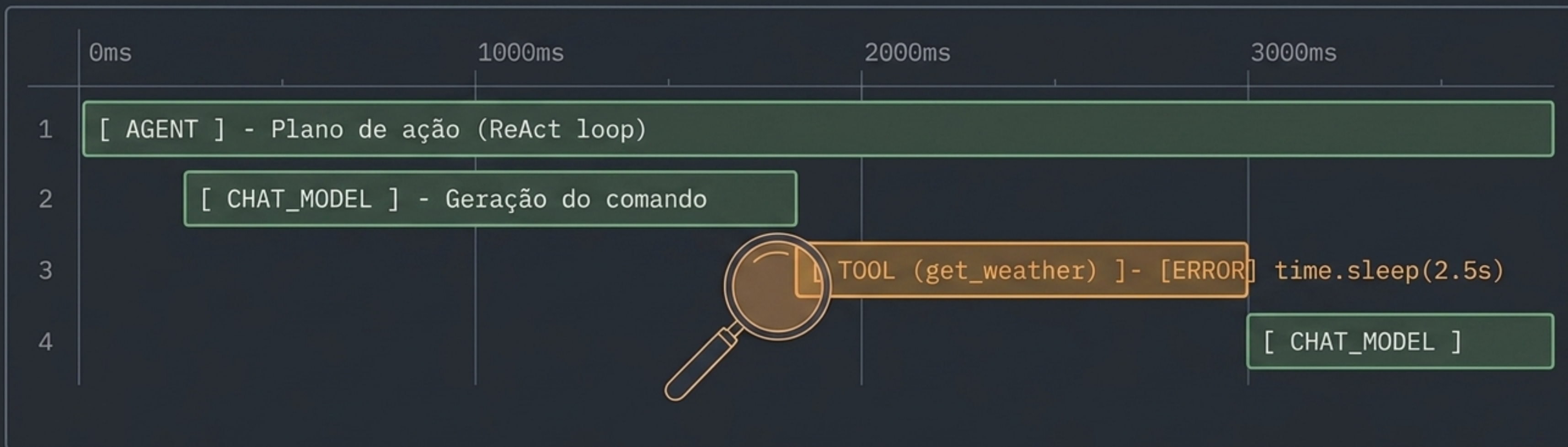
Latência

1.24s

Consumo

128 Tokens

Anatomia de um Trace Complexo (Onde o Agente Falhou?)



`mlflow gantt-xos` executa: execution com.
O decorator `@mlflow.trace` cria automaticamente árvores de
execução para pipelines complexos (RAG e LangChain).

2. Onde Está o Meu Dinheiro? (Custos & Tokens)

> make tokens



Cálculo automático e nativo para provedores suportados (OpenAI, Anthropic). Monitoramento em tempo real do uso de tokens por trace.



Gotcha Crítico: Modelos Self-Hosted

O MLflow não infere custos sozinho para modelos locais. É necessário injetar e setar o custo manualmente no span para evitar furos graves nas métricas financeiras.

3. Quem Vigia o Vigia? (Estratégias de Avaliação)

> make judges

0 Juiz LLM (0 Subjetivo)

+

0 Avaliador de Código (0 Objetivo)

Mecanismo: Usa um modelo poderoso (ex: GPT-4) para julgar as saídas.

Foco: Tom de voz, prestatividade, toxicidade.

Custo: Requer tokens adicionais, adiciona latência.

Mecanismo: Regra determinística em Python puro.

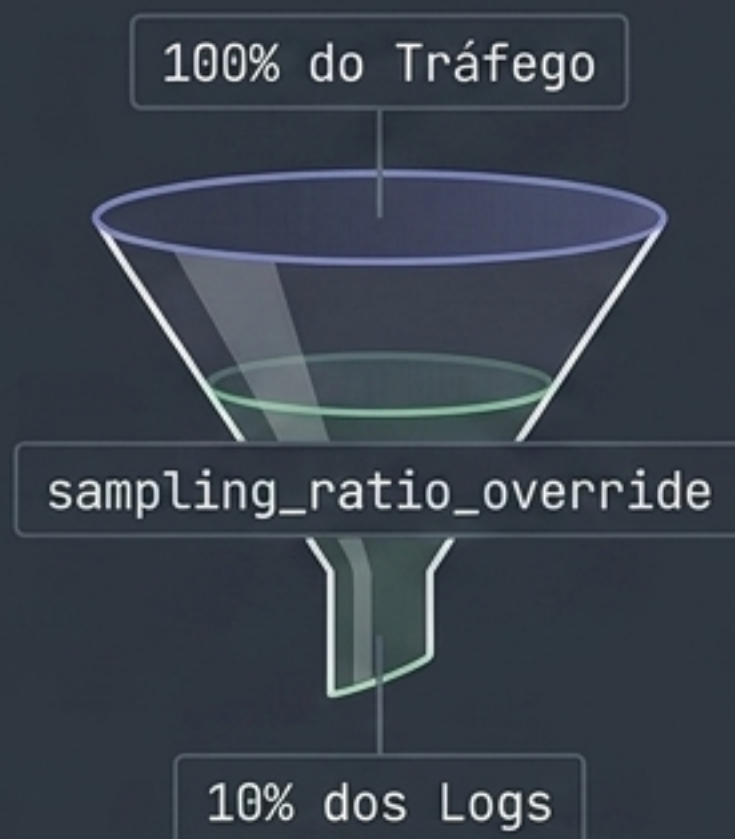
Foco: Contém um link válido? Formato JSON correto? Contém palavrão?

Custo: Latência Zero. Custo Zero.

A Dica de Ouro: Combine os dois. Use Python para aplicar regras estritas de graça, e reserve os LLMs para avaliar nuances complexas de linguagem.

4. Da Máquina Local para a Escala de Produção

> make monitoring



Sampling Inteligente:
Foque em auditorias de fluxos críticos
(ex: pagamentos) e amostre o restante para economizar storage.



0 Contexto da Sessão

Trace isolado não resolve suporte. Vincule sempre `user_id` e `session_id` para reconstruir interações multi-turn na hora de debugar.



Feedback Humano

0 verdadeiro diferencial.
Anexe a nota do usuário (value: 👍/👎) e justificativa escrita (rationale) diretamente ao Trace original.

A Arquitetura Invisível: Flush Assíncrono

> make traces



0 Problema: Por padrão, o logging de traces do MLflow é assíncrono para não impactar a latência da sua aplicação de IA. Tentar ler um trace via API imediatamente após a execução retorna um objeto vazio (None).

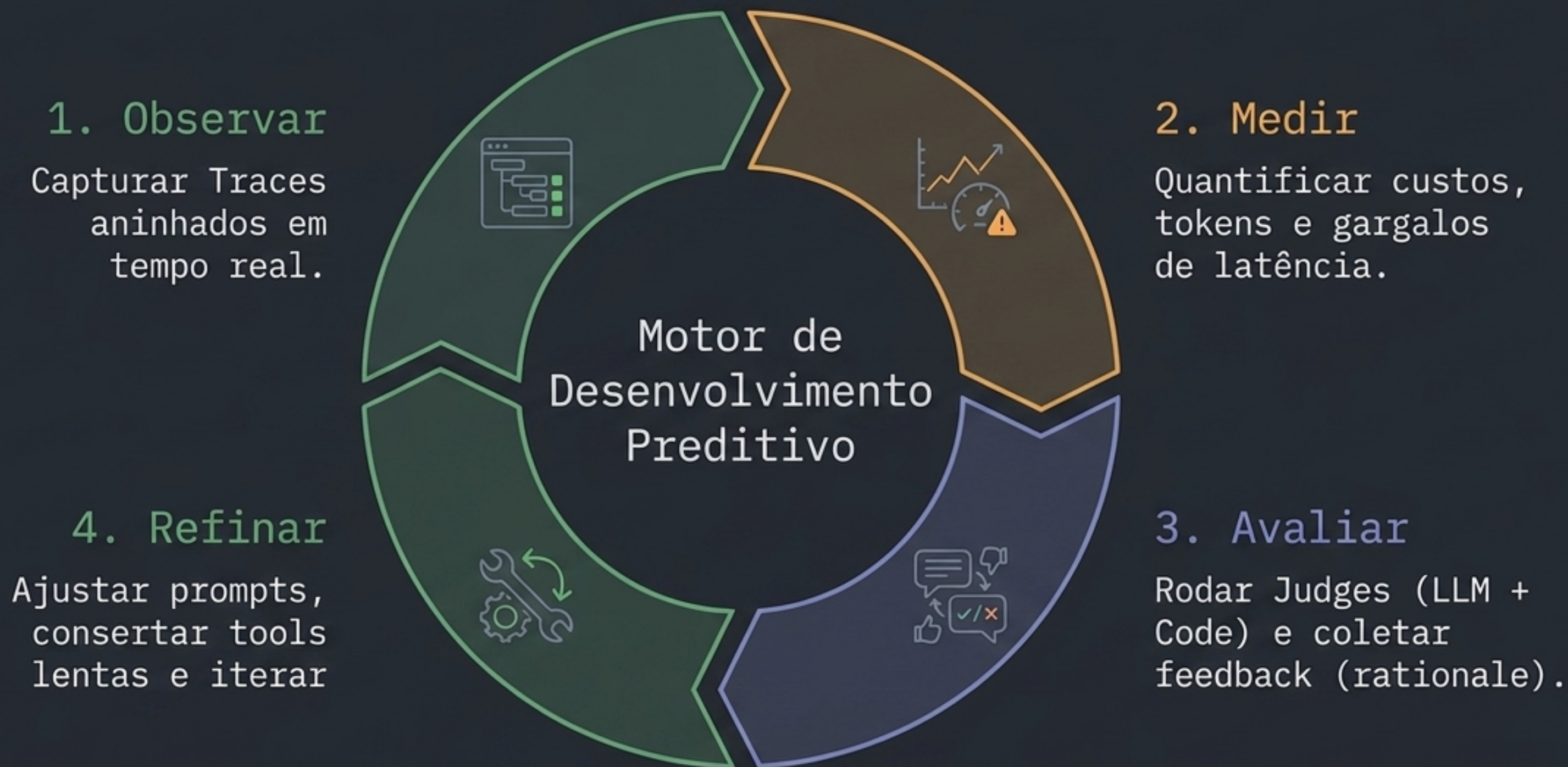
A Abordagem Incorreta

```
1  
2 model.invoke(input)  
3  
4 trace = client.get_trace(id) # RETORNA NONE!  
5
```

A Solução de Ouro

```
1 # Força o esvaziamento da fila assíncrona  
2 mlflow.flush_trace_async_logging()  
3  
4 # Agora é seguro buscar o trace  
5 trace = client.get_trace(id)
```


A Síntese: O Ciclo Virtuoso da Engenharia de IA



A observabilidade deixa de ser uma ferramenta de reação a incidentes e se torna o motor principal do seu desenvolvimento.

Checklist de Ouro

- [X] Ligue o auto-tracing com 1 linha (`mlflow.openai.autolog()`).
- [X] Estruture spans aninhados para expor falhas em fluxos RAG e Agentes.
- [X] Registre os custos de tokens na unha caso utilize modelos self-hosted.
- [X] Combine LLM Judges (para nuances) com Scorers de Código (latência zero).
- [X] Em produção, use sampling inteligente e conecte o `session_id`.
- [X] Lembre do `flush_trace_async_logging()` antes de consumir traces via código.

Próximos Passos: O Código é Seu

```
user@host:~/ia-observability$ # Clone o repositório e rode as
demonstrações ao vivo:

> make tracing           # Auto-tracing básico
> make tokens            # Custos e usage
> make judges            # LLM Judges vs Code Scorers
> make langchain-agent   # Agentes complexos, ReAct e Tools
> make monitoring        # Sampling, sessions e feedback humano
```

Todo o código deste workshop está contido em scripts isolados e reproduzíveis.
Teste, quebre e adapte para o seu stack.

Referências e Documentação Oficial



Escaneie para acessar o
repositório ia-observability.

- > MLflow GenAI Documentation
- > Tracing Quickstart Guide
- > LLM Judges & Scorers Implementation
- > Production Monitoring Strategies

[Sessão Aberta para Q&A] - Acesse os terminais e faça suas perguntas.